Docker and Kubernetes Notes

Introduction

Docker and Kubernetes are tools used to build, run, and manage applications in a consistent and scalable way.
Docker packages applications into containers. Kubernetes manages those containers in production.

Docker Fundamentals

What is Docker

Docker packages an application together with its dependencies and environment.
This package is called a **container**.

Images and Containers

- Image → blueprint of the application
- Container → running instance of the image

Basic Commands

```
docker build -t my-app .
docker run -p 3000:3000 my-app
docker ps
docker stop <container_id>
```

Dockerfile

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
CMD ["npm", "run", "dev"]
```

Steps:

1. select base image
2. copy files
3. install dependencies
4. define execution command

Why Docker is Used

- consistent environment
- isolation between applications
- easy deployment

Kubernetes Fundamentals

What is Kubernetes

Kubernetes manages containers at scale.
It handles:

- deployment
- scaling
- recovery
- networking

Core Concepts

Pod

- smallest unit in Kubernetes
- contains one or more containers

Deployment

- manages pods
- controls number of replicas

Service

- exposes pods
- enables communication

Kubernetes Example

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 2
  template:
    spec:
      containers:
      - name: app
        image: my-app
```

This configuration ensures that two instances of the application are running.

Basic Kubernetes Commands

```
kubectl get pods
kubectl apply -f deployment.yaml
kubectl delete pod <name>
```

Typical Workflow

1. write application
2. create Docker image
3. run container locally
4. deploy to Kubernetes
5. scale and monitor application

Summary

- Docker packages applications into containers
- Kubernetes manages containers in production
- containers ensure consistency
- Kubernetes ensures scalability and availability